

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FACULTAD DE CIENCIAS FÍSICO MATEMÁTICAS

Número de práctica:	PIA
Nombre de la materia:	Programacion Avanzada para Ciberseguridad
Nombres de los alumnos:	Jair Gonzalo Cortés Romero Estefani Jazmín Meléndez Magallanes
Nombre del tutor:	Perla Marlene Viera Gonzalez
Datos de entrega:	Monterrey, N. L. a 21 de mayo de 2026

Resumen

SecureWatch es una suite de herramientas desarrollada en C++ cuyo propósito es facilitar tareas de auditoría y análisis de seguridad en sistemas Windows y Linux. El proyecto integra dos componentes principales: un módulo de análisis de integridad de archivos y un módulo de monitoreo de tráfico de red. La herramienta fue diseñada siguiendo buenas prácticas de desarrollo, incorporando mecanismos de registro de actividades (logging), manejo de errores y generación de reportes para facilitar el análisis posterior.

El objetivo principal es proporcionar una solución sencilla que permita verificar la integridad de archivos ejecutables y observar comportamientos potencialmente maliciosos dentro de una red local. Además, el sistema registra las acciones realizadas por el usuario, lo que facilita la trazabilidad y el análisis dinámico durante pruebas o investigaciones.

La herramienta se diseñó para funcionar tanto en Windows como en Linux, recomendándole este último debido a la disponibilidad nativa de bibliotecas de captura de paquetes y herramientas de análisis de seguridad.

Diseño

La arquitectura del proyecto se divide en tres módulos principales:

Módulo Principal (Main)

El módulo principal actúa como punto de entrada de la aplicación. Su función consiste en presentar un menú interactivo al usuario y coordinar la ejecución de los diferentes componentes del sistema.

Además, este módulo implementa un mecanismo de auditoría mediante el archivo `security_audit.log`, donde se registran las acciones realizadas durante la ejecución. Este registro permite reconstruir la actividad del usuario y facilita el análisis de comportamiento del programa.

Módulo de Análisis de Seguridad

Este módulo se encarga de analizar archivos ejecutables y verificar su integridad. Para ello, el programa abre los archivos en modo binario y calcula un checksum basado en el contenido leído.

La finalidad de este proceso es detectar modificaciones no autorizadas, corrupción de archivos o posibles inyecciones de código. Si el checksum obtenido difiere del esperado, puede indicar que el archivo ha sido alterado o comprometido.

Módulo de Análisis de Red

El módulo de red utiliza la biblioteca `PcapPlusPlus` para capturar paquetes directamente desde las interfaces de red disponibles en el sistema.

Durante la captura, el sistema analiza el tráfico observado y recopila estadísticas

relacionadas con direcciones IP, protocolos utilizados y volumen de tráfico. También implementa reglas heurísticas básicas para detectar comportamientos potencialmente maliciosos, como escaneos de puertos o generación excesiva de tráfico desde una misma dirección IP.

Al finalizar la captura se genera un reporte con los hallazgos observados y posibles recomendaciones para mitigar la actividad detectada. Entre las mejoras planeadas se encuentra la generación automática de reglas de firewall o recomendaciones específicas para bloquear direcciones IP sospechosas.

Implementación

La implementación fue realizada completamente en C++ utilizando el estándar C++17.

Las principales dependencias utilizadas son:

- Windows: Npcap
- Linux: libpcap y PcapPlusPlus
- Compilador GCC/G++

La captura de tráfico se implementó mediante PcapPlusPlus, una biblioteca que abstrae las funcionalidades de captura de paquetes proporcionadas por libpcap y WinPcap/Npcap.

El módulo de análisis de archivos utiliza lectura binaria estándar mediante flujos de entrada (ifstream), permitiendo procesar ejecutables sin necesidad de herramientas externas.

La compilación con símbolos de depuración se realiza mediante el siguiente comando: `g++ -std=c++17 -g main.cpp analizador.cpp NetworkMonitor.cpp -o SecureWatch.exe -I/usr/local/include/pcapplusplus -IPcap++ -IPacket++ -ICommon++ -lpcap`

La ejecución de la herramienta se realiza mediante los siguientes comandos dependiendo del OS.

- Linux: `sudo ./SecureWatch.exe`
- Windows `SecureWatch.exe`

En caso de windows se podría ejecutar el programa con privilegios administrativos cuando sea necesario realizar captura de tráfico.

Pruebas

Las pruebas realizadas se enfocan en verificar el correcto funcionamiento de los dos módulos principales.

En el módulo de análisis de archivos se utilizaron ejecutables de prueba con diferentes contenidos para comprobar que el cálculo del checksum detectara modificaciones realizadas manualmente. Los resultados obtenidos mostraron que los cambios introducidos en los archivos eran reflejados correctamente por el mecanismo de verificación.

Para el módulo de red se realizaron capturas sobre interfaces reales utilizando tráfico generado por navegación web y herramientas de diagnóstico de red. Durante estas pruebas se verificó que el sistema fuera capaz de identificar direcciones IP activas, contabilizar paquetes y registrar eventos considerados sospechosos de acuerdo con las reglas implementadas.

Asimismo, se comprobó la generación correcta de los archivos de reporte y el funcionamiento del sistema de auditoría mediante la creación y actualización del archivo de logs.

Estas pruebas se hicieron en Kali Linux en una máquina virtual para su fácil reproducción y demostración del programa.

Análisis Estático

Como parte del proceso de evaluación se realizó un análisis estático del ejecutable utilizando Ghidra.

Exportaciones

Las exportaciones encontradas no presentaron comportamientos particularmente preocupantes. No se identificaron funciones comúnmente asociadas con malware, tales como mecanismos de inyección de DLL, evasión de entornos virtualizados o ejecución remota.

Se observaron símbolos relacionados con el procesamiento de direcciones y capas IPv4, tales como referencias a estructuras similares a `IPv4Address` o `IPv4Layer`. Estos elementos son consistentes con la funcionalidad esperada del módulo de monitoreo de red y no representan evidencia de actividad maliciosa.

Adicionalmente, la inspección manual de las funciones exportadas no reveló comportamientos anómalos ni rutinas ocultas.

Strings

Durante la revisión de cadenas de texto se identificaron algunos elementos relevantes:

-----BEGIN

-----END

Estas cadenas parecen formar parte de mensajes de salida o estructuras de texto utilizadas por bibliotecas internas.

También se detectó la cadena:

```
ascii command -v nmcli >/dev/null 2>&1 || { echo 'nmcli not installed'
```

La presencia de esta cadena sugiere que alguna biblioteca utilizada por el programa verifica la disponibilidad de herramientas de red del sistema operativo. Sin embargo, el análisis realizado no permitió asociar dicha cadena con funcionalidades ocultas o comportamiento sospechoso.

En general, las cadenas identificadas resultaron consistentes con las dependencias empleadas por la aplicación.

Ingeniería inversa

La ingeniería inversa se realizó mediante el uso de Ghidra para examinar la estructura interna del ejecutable.

La descompilación permitió identificar claramente los módulos principales descritos durante la etapa de diseño. Se observó una separación adecuada entre la lógica de análisis de archivos, el monitoreo de red y la interfaz principal del usuario.

El flujo de ejecución resultó sencillo de seguir debido a la presencia de símbolos de depuración durante las pruebas y a la estructura modular implementada. Asimismo, no se encontraron técnicas de ofuscación, empaquetamiento ni mecanismos destinados a dificultar el análisis.

La ausencia de código auto modificable, cifrado de cadenas o llamadas dinámicas sospechosas facilitó significativamente la comprensión del comportamiento del programa.

Riesgos y mitigaciones

Debido a la naturaleza del proyecto, los riesgos identificados son relativamente limitados.

Uno de los principales riesgos consiste en la necesidad de privilegios elevados para la captura de tráfico de red. Una ejecución incorrecta podría provocar errores de permisos o impedir el acceso a las interfaces de captura. Como mitigación, la documentación especifica claramente los permisos requeridos para cada sistema operativo.

Otro riesgo corresponde a la posibilidad de falsos positivos en la detección de actividad sospechosa. Dado que el sistema utiliza reglas heurísticas simples, ciertos comportamientos legítimos podrían ser clasificados erróneamente como anómalos. Este riesgo puede reducirse mediante el ajuste de umbrales y la incorporación de reglas más avanzadas.

Finalmente, el cálculo de checksum implementado constituye una verificación básica de integridad y no reemplaza mecanismos criptográficos robustos. Como mejora futura se propone incorporar algoritmos como SHA-256 para incrementar la

confiabilidad del análisis.

Conclusiones

SecureWatch demuestra la integración exitosa de técnicas de análisis estático e inspección de tráfico de red dentro de una única herramienta. La arquitectura modular implementada facilita tanto el mantenimiento como la ampliación futura de funcionalidades.

El módulo de análisis de integridad permite detectar modificaciones en archivos ejecutables, mientras que el componente de monitoreo de red proporciona visibilidad sobre la actividad observada en la red local y genera reportes útiles para tareas de auditoría.

Los resultados obtenidos durante las pruebas y el análisis estático indican que la aplicación presenta un comportamiento coherente con sus objetivos funcionales y no contiene indicadores evidentes de actividades maliciosas. Asimismo, la utilización de registros de auditoría y reportes contribuye a mejorar la trazabilidad y la capacidad de análisis posterior.

Como trabajo futuro se contempla la incorporación de algoritmos criptográficos más robustos, reglas avanzadas de detección de amenazas y mecanismos automáticos de mitigación mediante integración con sistemas de firewall y control de acceso.